Custom Commerce SaaS — Research and Phased Build Plan

Oct 8, 2026 · @Arad Workspace

Overview

We will build our own multi-tenant commerce platform for Bangladeshi SMEs on Cloudflare and Neon free tiers, borrowing proven designs from established commerce platforms instead of running one, so the platform costs nothing until traffic justifies paying.

Locked decisions

Area	Decision
Build approach	Custom code, TypeScript end to end. Designs borrowed from public commerce data models (Shopify, commercetools, Vendure, Medusa, Saleor)
Cost	$0 per month at launch. Every component must run on a free tier; paid upgrades only when a measured limit is near
Hosting	Cloudflare (Workers, static assets, R2, Cron Triggers, KV) plus Neon Postgres. No always-on servers, no VMs
Tenancy	One codebase, one database. Every tenant-owned row carries `tenant_id`, enforced in code and by Postgres row-level security
Market	Bangladesh: BDT only, Bangla and English, cash on delivery, SSLCommerz, local couriers, SMS, phone-first shoppers
Sales model	We onboard clients (no self sign-up). Clients' payments settle into their own merchant accounts

Area	Decision
Growth rule	Start small, but every module has a clean boundary and a data model that can grow (more payment methods, couriers, currencies, a dedicated database per large client) without rewrites

MVP scope (what the first client gets)

- A branded storefront on a subdomain, later on their own domain, with Meta Pixel, Google Analytics and WhatsApp and Messenger chat buttons.
- Products with variants, images, stock and Bangla/English text.
- Cart, phone-first guest checkout, cash on delivery, then SSLCommerz.
- Orders with a clear status flow, delivery-zone charges, discounts, coupons and free-delivery offers, plus returns, exchanges, failed deliveries and refunds.
- Email and SMS order updates.
- A mobile-first web admin, installable on phones, with a sales dashboard, manual order entry and instant new-order alerts, scoped to the client's store only.

Explicitly out of scope until needed: multi-currency, complex tax engines, B2B pricing, marketplaces, subscriptions, native apps, self-service sign-up.

Timeline: the MVP (Phases 0–5B) is roughly 5 months for one developer working with an AI coding agent, followed by a 4–6 week pilot with two or three clients. These are estimates; each phase plan refines them.

Research: how commerce platforms are built

Established platforms split commerce into the same dozen domain modules, each owning its own data and talking to others through clear interfaces, and they share a small set of engineering patterns. We copy that structure and build only the parts Bangladeshi SMEs need.

The modules every platform has

Medusa ships its core as separate commerce modules (API key, auth, cart, currency, customer, fulfillment, inventory, order, pricing, product, promotion, and more), each defining its own tables. Vendure, Saleor, Shopify and commercetools use the same domains under different names. Our version:

Module	Owns	Approach borrowed	MVP
Tenancy	Stores (tenants), domains, settings, themes, plans	Vendure channels and Medusa sales channels: one record per storefront, everything scoped to it	Yes
Identity and access	Staff users, shopper accounts, sessions, roles, permissions	Separate staff and shopper identities, each a user with one account row per login method, so new methods need no schema change; staff roles scoped per store	Yes
Catalog	Products, variants, options (size, color), categories, collections, translations, media	Product holds shared content; each sellable combination is a variant with its own SKU, price and stock	Yes
Pricing	Variant prices, compare-at prices; later price lists and customer-group prices	Prices stored per variant in integer minor units; price lists added later as overrides	Yes (simple)
Inventory	Stock per variant per location, reservations, movement ledger	Medusa's model of stock across locations with reservations; every change is a ledger entry	Yes (one location)
Cart	Active baskets, line items, applied coupons, computed totals	Server computes all totals; the client never sends prices	Yes
Checkout	Address, delivery option, payment selection, order placement	A single orchestrated step that validates everything and creates the order atomically	Yes

Module	Owns	Approach borrowed	MVP
Orders	Placed orders, line snapshots, status history, edits, cancellations	Vendure's finite state machine where each state allows only defined next states	Yes
Payments	Payment records, provider handlers, refunds	Pluggable provider handlers (COD, SSLCommerz, later bKash direct) behind one interface	Yes
Fulfillment and shipping	Delivery zones and rates, fulfillments, courier bookings, tracking	Vendure allows several fulfillments per order, each with its own state machine and pluggable handler	Yes (manual shipments; couriers later)
Promotions	Coupons, automatic discounts, free delivery	Rules made of conditions (minimum spend, product, customer) plus actions (percent off, fixed off, free delivery)	Yes
Customers	Accounts, guest records, addresses, groups	Guest checkout by phone; accounts optional; addresses owned by the customer within one store	Yes
Tax	VAT rates, prices with or without VAT	Tax-inclusive pricing with one rate per store to start	Simple
Returns	Return requests, exchanges, failed deliveries, restock, refunds, store credit	Return or exchange against order lines with reasons and inspection; restock by condition; refund or store credit; failed deliveries treated as returns to origin	Yes

Module	Owns	Approach borrowed	MVP
Notifications	Email, SMS, WhatsApp messages and templates	Event-driven sending from a background queue, never inside the request	Yes
Search	Product search and filters	Postgres trigram search (pg_trgm), which works for Bangla and tolerates typos; a dedicated engine later	Basic

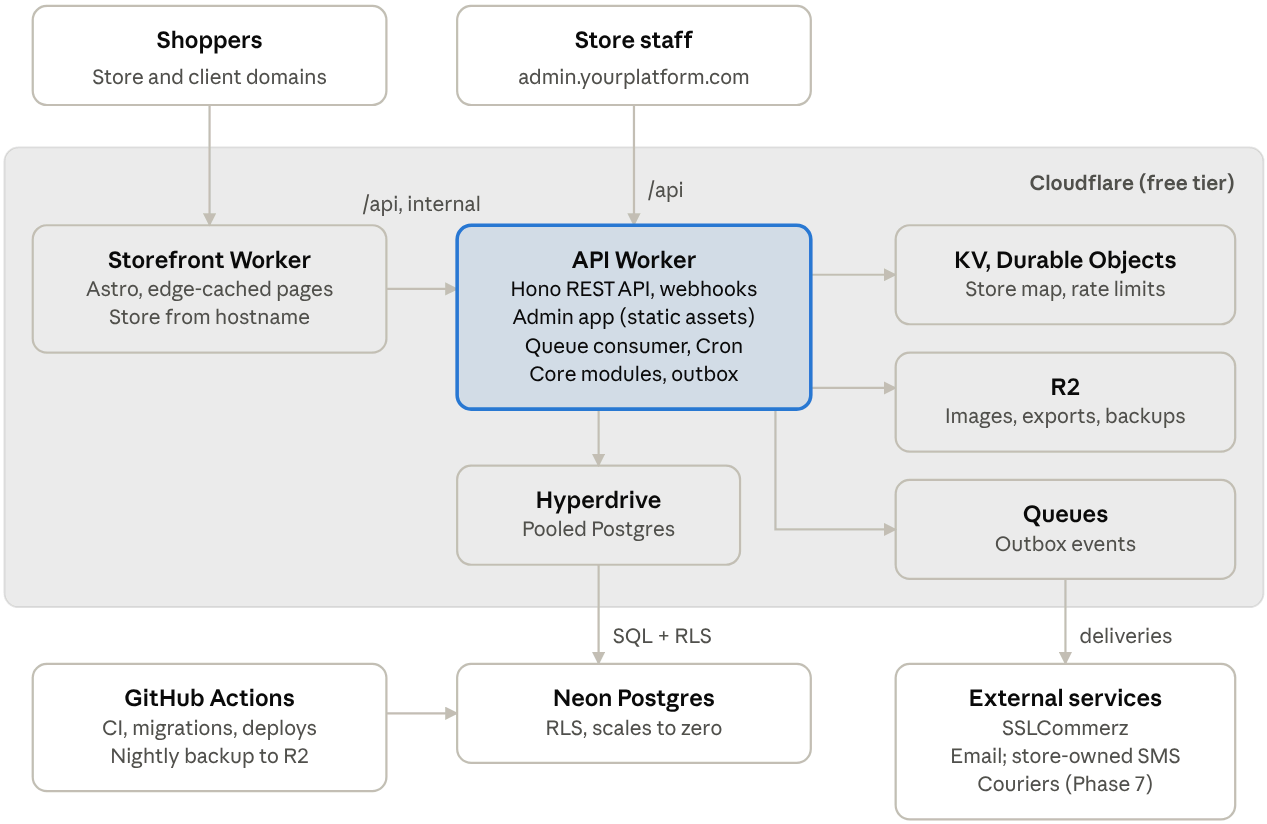
Patterns every serious platform uses

- **State machines for anything with a lifecycle.** Orders, payments, fulfillments and returns each have a fixed set of states and allowed transitions. Illegal transitions are rejected in one place, and every transition is recorded.
- **Cart and order are separate.** Vendure models a cart as an order that has not finished checkout. We separate them, as Medusa does: carts are disposable, and an order is created once at checkout with frozen copies of names, prices, discounts and addresses, so later product edits never change past orders.
- **Pluggable providers.** Payment, courier, SMS and email integrations sit behind small interfaces with one adapter per vendor, so adding bKash or a new courier means a new adapter, not a change to order logic.
- **Idempotency on every write that can be retried.** Stripe's approach: the client sends a unique key, the server stores the first result for that key, and retries return the same result instead of repeating the action. We apply this to order placement, payments and webhooks.
- **Transactional outbox for side effects.** Business changes and the event describing them are written in the same database transaction; a separate process delivers the events (emails, SMS, courier sync) with retries. An order never fails because an SMS provider is down.
- **Shared database, tenant key plus row-level security.** Every tenant row carries `tenant_id`, and Postgres policies filter by a transaction-local tenant setting, so a missing `WHERE` clause returns nothing instead of another tenant's data.
- **Modular monolith.** One deployable API, but code organized by module with no module reaching into another's tables. This is how Medusa and Vendure stay extensible without microservices.

Architecture

The platform is two Cloudflare Workers and one Neon database: the storefront Worker serves cached pages, and the API Worker holds all commerce logic, the admin app, background jobs and scheduled tasks.

Two Workers and one database, all on free tiers



platform architecture · two Workers, Neon, Cloudflare services

Shoppers mostly hit edge-cached storefront pages; only cart, checkout and admin calls reach the API Worker, which talks to Neon through Hyperdrive with the tenant set on every transaction. Side effects leave through the outbox and Queues, so a slow SMS or payment provider never blocks an order.

Domains and routing

Two domains keep the product brand safe: a main domain for us and client staff, and a separate, swappable store domain for everything shoppers see. Placeholders below: `yourplatform.com` (main) and `yourstores.com` (stores).

Address	Served by	Purpose
`yourplatform.com`	Static site on Workers	Company and marketing site
`admin.yourplatform.com`	API Worker	Admin app for client staff; super-admin pages under `/platform`
`admin.yourplatform.com/api/*`	API Worker	Admin, platform and login API (maps to `/v1/admin/*` and `/v1/platform/*`)
`api.yourplatform.com`	API Worker	Webhooks only: SSLCommerz, couriers, later Meta
`{slug}.yourstores.com`	Storefront Worker	Default store address; setup and test stores
`www.clientbrand.com`	Storefront Worker	Client's own domain through Cloudflare for SaaS
`{store host}/api/*`	Storefront Worker, forwarded to the API Worker by a service binding	Cart, checkout, order tracking, shopper login (maps to `/v1/store/*`)
`{store host}/media/*`	Storefront Worker, reading from R2	Product images, logos, banners
`{store host}/sitemap.xml`, `/robots.txt`, `/feeds/meta.xml`	Storefront Worker	Per-store SEO files and catalog feed
`stores.yourstores.com`	Cloudflare for SaaS fallback origin	CNAME target for client domains
`*.workers.dev`	Workers	Preview environments only

Rules

- Nothing a client controls appears on the main domain: no stores, uploads, images or shopper emails.
- Shopper requests stay on the store's own host, so cart and session cookies are first-party and no CORS is needed.
- The admin app and its API share one host, so the staff login cookie is first-party too.
- The store domain is one configuration value (`STORE_DOMAIN`), never hardcoded in code, templates or emails.
- Reserved slugs that no client can take: `admin`, `api`, `www`, `media`, `status`, `mail`, `help`, `stores`, `app`, `cdn`, auth, plus offensive or brand-confusing names. auth.yourstores.com is kept for shared social-login sign-in.
- Email: shopper emails (orders, returns) are sent from the store domain with the store's name as the sender; our own emails to clients (invoices, staff login codes) come from the main domain. SPF, DKIM and DMARC on both.
- Monitoring and abuse: both domains in Google Search Console with alerts, auto-renew on, a monitored abuse address, the suspend action for fast takedown, and terms that ban prohibited products.

If the store domain is ever flagged

1. Register a new store domain and add it to Cloudflare (wildcard route, Cloudflare for SaaS fallback origin, email records).
2. Change `STORE_DOMAIN` and deploy. Default store addresses and the shopper email sender move to the new domain.
3. Keep the old domain registered, so client domains that point to it keep working. Clients change nothing.

Setup takes about a day once; ongoing maintenance is close to zero.

Core engineering principles

These rules apply to every module and every phase; the agent should treat them as non-negotiable unless an ADR changes them.

Tenancy and isolation

- Every tenant-owned table has `tenant_id` (non-null, foreign key, first column of every composite index).

- Two layers of protection: the data-access layer always adds the tenant filter and refuses to run without a tenant context, and Postgres row-level security with `FORCE ROW LEVEL SECURITY` enforces the same rule in the database.
- Tenant context is set per transaction with `set_config('app.tenant_id', …, true)`, never per session, because connections are pooled.
- The application connects with a restricted role that cannot bypass RLS. Migrations and platform-admin jobs use a separate role.
- Global reference data (divisions, districts, areas, currencies) lives in non-tenant tables and is read-only to tenants.

Money, numbers and time

- Money is an integer in minor units (poisha) with a currency code column, even though only BDT exists today. No floats anywhere in money math.
- Rounding happens once, at defined points (line total, order total), with a documented rule.
- All timestamps are UTC (`timestamptz`); display in Asia/Dhaka.
- Phone numbers are stored in one international format (E.164, for example `+8801712345678`) and shown in local format; every input (checkout, import, admin, manual orders) is normalized before saving or matching.

Identifiers

- Internal IDs: UUIDv7 (time-ordered, index-friendly, safe to expose).
- Human-facing codes: short, per-tenant order numbers (for example `ORD-10234`) generated by a per-tenant sequence, never the internal ID.
- Variant SKUs unique per tenant.

Lifecycles

- Orders, payments, fulfillments and returns each have an explicit state machine defined in code as data (states, allowed transitions, guards, side effects).
- Every transition writes a history row (from, to, actor, reason, time).
- Orders keep frozen snapshots of product names, variant options, unit prices, discounts, delivery fee and address at the time of purchase.

Consistency

- Each command (place order, record payment, adjust stock) runs in one database transaction.

- Stock is changed only through an inventory service that writes a movement ledger row; on-hand quantity is derived or kept in sync inside the same transaction. Reservations are taken at order placement and released on cancellation.
- Concurrency on hot rows (stock, coupon usage) uses conditional updates (`UPDATE … WHERE available >= qty`) or row locks, never read-then-write in application code.
- Writes that clients may retry (checkout, payment callbacks, admin actions) accept an `Idempotency-Key` and store the first result.

Events and background work

- Every meaningful change emits a domain event (for example `order.placed`, `payment.settled`, `fulfillment.shipped`) written to an `outbox` table in the same transaction.
- A dispatcher moves outbox rows to Cloudflare Queues; consumers send email and SMS, sync couriers and refresh caches. Consumers are idempotent because delivery is at least once.
- Scheduled work (reservation expiry, cleanup, report rollups) runs once a day in a single Cron Trigger batch; work due later (such as checking a pending payment) uses delayed queue messages. Nothing polls the database on a short timer.
- Each queue has a dead-letter queue: messages that still fail after their retries are kept, raise an alert, and can be retried from the platform dashboard.

Extensibility

- Providers (payment, courier, SMS, email, storage) are interfaces with one adapter per vendor, chosen per tenant by configuration.
- Per-tenant settings and feature flags live in typed JSON validated by zod schemas, so new options do not need migrations.
- Custom attributes on products and orders use a typed `metadata` JSON column, never ad-hoc columns per client.

Auditability

- An `audit_log` records every staff action on settings, prices, stock, orders and payments: who, what, before and after, when.

Repository, tooling and environments

One pnpm + Turborepo monorepo with only two deployable Workers (API and storefront), so there is almost nothing to operate.

Stack

Layer	Choice	Why
API	Hono on Cloudflare Workers, REST with OpenAPI generated from zod schemas	Small, fast, Workers-native; REST is cacheable and cheap on CPU
Database access	Drizzle ORM over Cloudflare Hyperdrive to Neon Postgres	Typed SQL, real transactions (needed for row-level security), connection pooling at the edge
Auth	Better Auth as two separate instances with the Drizzle adapter: staff (organization, two-factor and admin plugins) and shoppers (email and phone code plugins; social providers and passkeys later)	One well-tested library for every login method; each side grows by switching on plugins, and its user-plus-accounts model needs no schema change for new methods. Verify on Workers in Phase 0
Storefront	Astro with the Cloudflare adapter, small interactive islands for cart and checkout	Very little JavaScript for low-end phones, strong SEO, first-class Workers support
Admin dashboard	React + Vite single-page app (TanStack Router and Query, shadcn/ui), served as static assets by the API Worker	Static asset requests are free; no separate deployment
Background work	Cloudflare Queues consumer and Cron Triggers in the API Worker	Free tier, same codebase, same deploy
Files	Cloudflare R2	Free tier, no egress fees
Lint and format	Biome	One fast tool
Tests	Vitest (unit and integration against real Postgres), Playwright (end to end)	Standard, fast

Layout

```
/apps
  /api            Hono app: routes, modules, queue consumer, cron; also
serves /admin assets
  /admin          React SPA (built into apps/api static assets)
  /storefront     Astro storefront, multi-tenant, themeable
/packages
  /db             Drizzle schema, SQL migrations, RLS policies, seed scripts
  /core           Domain modules: tenancy, catalog, pricing, inventory, cart,
checkout,
                  orders, payments, fulfillment, promotions, customers,
notifications
  /providers      Adapters: sslcommerz, cod, steadfast, pathao, redx, sms-*,
email-*
  /shared         zod schemas, money utils, state machine helper, error
types, i18n keys
  /sdk            Typed API client generated from the OpenAPI spec
  /ui             Design tokens and shared components
/docs             ADRs, runbooks, module docs
```

Domain logic lives in `packages/core`, independent of Hono and Workers, so it is unit-testable and portable if hosting ever changes.

Environments (all free)

Environment	API and storefront	Database	Purpose
Local	`wrangler dev`	Postgres in Docker	Daily development
Preview	Workers preview versions per pull request	Postgres container in CI; Neon branch only when labeled	Review
Production	Workers on the platform domain and client domains	Neon main branch	Live clients

CI/CD (GitHub Actions)

1. Pull request: install, Biome check, type-check, unit and integration tests (including the isolation suite), build, deploy preview, run Playwright smoke tests against it.
2. Merge to `main`: apply migrations to production with the migration role, then deploy both Workers. Keep the previous Worker version for one-command rollback.

3. Migrations are generated by drizzle-kit, reviewed in the pull request, backward compatible (expand, then contract) so a rollback of code never breaks on the new schema.

The repository lives on a personal GitHub account on the free plan, where branch protection is not available for private repositories. So the deploy job depends on the test job (a failing commit on `main` never deploys), and changes reach `main` only through pull requests whose checks pass. GitHub Pro or a move to an organization can add enforced protection later.

Secrets and configuration

- Worker secrets via `wrangler secret`; GitHub Actions secrets for CI. Never in the repository.
- Configuration validated by a zod schema at startup.
- Per-tenant credentials (SSLCommerz, courier, SMS) encrypted with AES-GCM using Web Crypto and a key held as a Worker secret.

Prerequisites by phase

Everything below comes from outside the codebase (accounts, domains, credentials, business decisions, legal documents), so the product owner provides it before the phase starts; items with long lead times should be started weeks earlier.

Start these now (long lead times)

- Company trade license and bank account: needed for our own SSLCommerz merchant account and Meta business verification.
- Our SSLCommerz merchant account (for collecting subscription fees) and each pilot client's own merchant account: approval can take weeks and needs a trade license, bank account and a working website.
- Legal review: platform terms, privacy policy, data processing agreement, store legal page templates, and an accountant's view on record retention and VAT.
- A `.com.bd` domain, if chosen: registration needs local paperwork and takes longer than a `.com`.
- Meta business verification and Tech Provider application (Phase 9): can take weeks.

Phase	What to prepare before it starts
0 — Foundations	Platform name; accounts on GitHub (a personal account with a private repository is fine), Cloudflare, Neon (region closest to Bangladesh), Sentry and an uptime monitor. No domain is needed yet: development uses localhost and previews use workers.dev addresses. Buying the main domain early is still recommended, because email providers trust older domains more
1 — Platform core	Main domain registered, with nameservers on Cloudflare and Google Search Console access (needed by the end of this phase); transactional email provider account with the main domain verified (SPF, DKIM, DMARC) for staff sign-in codes; confirmation of the staff roles (Owner, Manager, Staff, Viewer) and what each may do; the first super-admin email addresses
2 — Catalog and inventory	A real product spreadsheet from a pilot client (variants, prices, stock, image links, Bangla and English text) for import testing; approval of the division, district and area dataset the agent proposes
3 — Storefront and domains	Store domain registered, with nameservers on Cloudflare and Google Search Console access; platform logo and brand colors; two or three reference stores per theme preset (for example fashion, grocery, general); a pilot client's logo, colors and banners; Cloudflare for SaaS enabled on the store domain; a test Meta Pixel ID, Google Analytics 4 property, WhatsApp number and Messenger page link; cookie consent wording
4 — Cart, checkout, orders	Pilot clients' delivery zones and fees, COD rules (maximum order value, advance delivery charge), promotions they usually run, and what must appear on invoices and packing slips; email provider verified for the store domain (shopper sign-in codes)

Phase	What to prepare before it starts
5A — Payments, notifications, returns	SSLCommerz sandbox credentials; the one or two SMS gateways to support first, with a test account and API credentials; pilot clients' return policies and refund methods; approved Bangla and English wording for order, return and refund messages
5B — Launch readiness and pilot	Final plans, prices and limits; our invoice details (company name, address, VAT registration if any); our SSLCommerz merchant account and bKash or bank details for subscription payments; signed legal documents; support email, support WhatsApp number and abuse address; a backup encryption key stored safely offline; two or three signed pilot clients with their live SSLCommerz accounts; a Bangla help guide for clients (short articles or videos on using the admin)
6 — Sales recovery	Pilot feedback on fake orders and confirmation calls; the store's confirmation call script and the score thresholds that should trigger an advance delivery charge
7 — Couriers	API credentials for Steadfast, Pathao and RedX (sandbox where offered), using our test accounts or pilot clients' merchant accounts; each courier's webhook or status-polling details
8 — Back office	Sample supplier, purchase and expense data from a pilot client; an accountant's review of the profit and stock valuation reports
9 — Meta and messaging	Verified Meta Business account; a Meta app with WhatsApp Cloud API access; Tech Provider approval; pilot clients' WhatsApp Business numbers; approved WhatsApp message templates
Later features	Google Cloud project and OAuth consent screen, and a Facebook app with Facebook Login approved, for shopper social login; our own SMS gateway account for platform SMS

Phase 0 — Foundations

Goal: a deployable skeleton on free tiers, with the riskiest technical assumptions proven before any feature work. Rough size: 1–2 weeks.

Spikes first (each ends with an ADR)

- [ ] **Database path:** Drizzle over Hyperdrive to Neon, with an interactive transaction that sets `app.tenant_id` and reads through RLS. Measure latency from Bangladesh-facing Cloudflare locations; pick the Neon region closest to Bangladesh that is available. Fallback: Neon's serverless driver over WebSockets.
- [ ] **CPU budget:** confirm a typical API request (auth check, 3–5 queries, JSON response) and a server-rendered Astro product page stay under the free plan's 10 ms CPU per request. If not, identify what to cache or move to the paid plan.
- [ ] **Bundle size:** confirm each Worker stays under the free plan's 3 MiB compressed limit.
- [ ] **Auth:** two Better Auth instances on Workers with Drizzle. Staff: `platform` schema, organization plugin for store membership and roles. Shoppers: tables in the `store` schema with `tenant_id`, email and phone unique per store, and RLS so the instance only ever sees the current store; prove that two shoppers with the same email in two stores stay fully separate. Measure password-hashing CPU time against the free plan's 10 ms limit; if it does not fit, staff sign in with email codes and passkeys until the paid plan.
- [ ] **Edge state on the free plan:** KV allows only 1,000 writes a day on the free plan, so use it only for rarely changing data (domain-to-store map, cache versions). Rate-limit counters use the Workers rate-limiting binding or a Durable Object; turn off Astro's default KV session storage. Confirm each choice works on the free plan.
- [ ] **Per-store cache invalidation:** test purging one store's cached pages on the free plan. Fallback: a per-store catalog version in every cache key, bumped on change (debounced, so a bulk import causes one write), so old pages simply stop being used.

Build

- [ ] Monorepo, Biome, TypeScript strict, Turborepo, shared tsconfig.
- [ ] `packages/db`: base schema (tenants, outbox, audit log, idempotency keys), migration and app database roles, RLS helper to generate policies per table, seed script with two demo tenants.
- [ ] `packages/shared`: money type and helpers, UUIDv7, state machine helper, typed error model (problem+json responses), i18n setup with Bangla and English.

- [ ] `apps/api`: Hono app with request ID, structured JSON logging, error handler, zod validation, OpenAPI document at `/openapi.json`, health route, queue consumer and cron handler stubs, static assets for `/admin`.
- [ ] `apps/storefront`: Astro on Workers with a placeholder page that resolves the tenant from the hostname.
- [ ] CI/CD as described: tests against a Postgres container in GitHub Actions; a Neon preview branch only when a pull request is labeled for it, deleted after merge. Neon compute fixed at 0.25 CU; Cloudflare bot protection on both domains; R2 lifecycle rules for exports, temporary uploads and backups.
- [ ] Monitoring on free tiers: Workers Logs and analytics, Sentry (free plan) for errors in API, admin and storefront, an external uptime check on `/health`.
- [ ] Development without a domain: `{slug}.localhost` hostnames for testing several stores locally, previews on `workers.dev` addresses with a development-only store selector that is switched off in production, and a local mail catcher for email codes. Domain setup happens when each domain arrives: the main domain by the end of Phase 1, the store domain by Phase 3.

Acceptance criteria

- A pull request gets its own preview API and storefront automatically, and a database branch when labeled for it.
- The isolation test harness exists and runs in CI (it will grow with every module).
- Spike ADRs record the measured CPU time, latency and bundle sizes.

Phase 1 — Platform core

Goal: tenants, staff accounts, roles and the admin shell, with isolation proven, before any commerce data exists. Rough size: 2 weeks.

Tenancy module

- [ ] `tenants`: slug, name, status (active, suspended, closed, purged), default locale, settings (typed JSON), plan.
- [ ] `tenant_domains`: hostname, type (platform subdomain or custom), verification status, primary flag.
- [ ] Tenant resolution: hostname to tenant via a KV cache (refreshed by an outbox event on change) with database fallback. Admin and API requests resolve the tenant from the authenticated staff member's active store, never from client input alone.

- [ ] Platform CLI and super-admin API: `tenant:create` provisions the tenant, its owner account, default settings and subdomain in one transaction.

Data ownership (prepares for clean tenant removal)

- [ ] Two classes of data in separate Postgres schemas. **Store data** (`store` schema): everything a client creates or collects, such as catalog, customers, orders, returns, files and credentials; every table has `tenant_id` and RLS. **Platform data** (`platform` schema): our own records about each client, such as the tenant registry, plan and billing history, contract dates, platform audit log and purge records; cleanup never touches it.
- [ ] A tenant table registry generated from the `store` schema, with a CI test that fails if any table is missing from the purge order.
- [ ] Every tenant file stored under `tenants/{tenant_id}/` in R2, and every KV key prefixed with the tenant ID.

Identity and access

- [ ] Staff users with email and password, email codes for verification and password reset, and optional two-factor login, on the Better Auth staff instance (Google login and passkeys can be switched on later); one user may belong to several stores (agency staff) with a role per store.
- [ ] Roles: Owner, Manager, Staff, Viewer, each a set of fine-grained permissions (for example `catalog.write`, `orders.read`, `payments.refund`, `settings.write`). Permissions are checked in the route layer and again in the core service.
- [ ] Platform roles (super admin, support) kept separate from tenant roles, with two-factor login required for them; support access to a store is time-limited and audited.
- [ ] Security: rate limits on login and OTP (Workers rate-limiting binding or a Durable Object counter, never KV), lockout after repeated failures, session expiry and rotation, `HttpOnly` `Secure` `SameSite=Lax` cookies, CSRF protection for cookie-authenticated writes.
- [ ] Basic email sending behind an `EmailProvider` interface: the platform's default sender, from the main domain for staff emails (sign-in codes, password reset, invitations) and from the store domain for shopper emails once Phase 4 needs them. Phase 5A adds per-store providers, order notifications and editable templates on top.
- [ ] Connect the main domain: Worker routes for the admin and webhook hosts, the email sender verified with SPF, DKIM and DMARC, and Search Console alerts on.

Admin shell

- [ ] Login, store switcher, navigation, settings pages for store profile and staff management, Bangla and English UI.
- [ ] Shared table, form and dialog components used by every later module.
- [ ] Mobile-first admin: every screen designed for a phone first, then widened for desktop; installable as a Progressive Web App (home-screen icon, full-screen, web push support) at no cost.
- [ ] Admin meets WCAG 2.1 AA for contrast, form labels and keyboard use.

Acceptance criteria

- Isolation suite: a user of store A gets 404 on every store B resource by ID, and raw SQL as the app role with tenant A context returns zero store B rows.
- Creating a tenant with the CLI takes one command and produces a working admin login and subdomain.

Phase 2 — Catalog, media and inventory

Goal: clients can build their full catalog in Bangla and English with variants, prices, images and stock. Rough size: 3 weeks.

Catalog

- [ ] Products: title, slug, description (rich text stored as sanitized structured content), status (draft, active, archived), SEO fields, category and collection links, metadata.
- [ ] Options and option values (size, color) per product; variants generated from option combinations, each with SKU, barcode, price, compare-at price, cost price, weight, stock tracking flag.
- [ ] Translations stored in per-locale translation tables for product, variant option labels, category and collection text, with a fallback locale.
- [ ] Categories (tree, for navigation) and collections (manual or rule-based, for merchandising).
- [ ] Slugs unique per tenant and locale; redirects table so renamed slugs keep working.
- [ ] VAT settings per store: VAT on or off, the rate, and whether prices include VAT; a VAT category per product where needed. VAT is calculated and stored per order line and shown on invoices.

Media

- [ ] The admin app resizes photos in the browser to WebP at two or three fixed widths, then uploads them to R2 through short-lived signed URLs; the server validates type and size and stores dimensions. Originals are not kept.
- [ ] Files named by content hash (duplicates stored once, cacheable for a year); a weekly job removes images no product uses.
- [ ] Ordered images per product and optional image per variant.

Inventory

- [ ] Stock locations (one default per tenant now; the model allows many).
- [ ] Inventory levels per variant per location: on hand, reserved, available (derived).
- [ ] `stock_movements` ledger: type (receive, sale, reservation, release, adjustment, return), quantity, reason, reference (order, purchase), actor. Every change goes through the inventory service.
- [ ] Low-stock threshold per variant; allow-backorder flag.

Admin and import

- [ ] Product list with search and filters, product editor with variants grid, bulk price and stock edit.
- [ ] CSV and Excel import and export (products, variants, prices, stock, image URLs) with a dry-run preview and row-level error report; run as a queued job.

Acceptance criteria

- A client imports 500 products with variants from a spreadsheet in one go, and the catalog appears correctly in both languages.
- Every stock change is traceable in the ledger, and on-hand equals the sum of movements for every variant (a test enforces this).

Phase 3 — Storefront, themes and domains

Goal: every tenant gets a fast, branded, SEO-friendly store from one Astro codebase, with themes chosen by configuration. Rough size: 3 weeks.

Rendering and caching (the key to staying free)

- [ ] Catalog pages (home, category, collection, product) are server-rendered once and cached at Cloudflare's edge per tenant and locale; cache is invalidated by outbox events when products, prices or stock change (purge or version bump, as decided in Phase 0). Most shopper traffic then never reaches the API or the database.

- [ ] Cart, checkout and account parts are small client-side islands calling the API directly; never cached.
- [ ] A public, read-only catalog endpoint set in the API designed for caching (stable URLs, cache headers, ETags).

Themes

- [ ] A theme is a named template set plus design tokens (colors, fonts, radius, logo, banners, homepage sections) stored in tenant settings and validated by a zod schema.
- [ ] Start with one theme and two or three homepage layouts; adding a theme later means adding a folder of templates, not changing the core.
- [ ] Theme settings editor in the admin with live preview.

Pages and SEO

- [ ] Home, category, collection, product, search, cart, checkout, order confirmation, order tracking (order number + phone), account, static pages (about, contact, policies).
- [ ] Per-tenant sitemap, robots, canonical URLs, Open Graph, schema.org Product and Offer data, hreflang for Bangla and English.
- [ ] Performance targets on a mid-range Android phone over 4G: LCP under 2.5 s, INP under 200 ms, CLS under 0.1; JavaScript on product pages under about 50 KB compressed.
- [ ] Tracking settings per store: Meta Pixel ID and Google Analytics 4 ID; browser events for page view, product view, add to cart, checkout started and purchase, with a cookie consent banner. Server-side Meta Conversions API follows in Phase 9.
- [ ] Floating WhatsApp and Messenger chat buttons, configured per store (number or page link, greeting text, show or hide by page).
- [ ] Product search using Postgres trigram matching (`pg_trgm`) on Bangla and English names, SKUs and tags, tolerant of typos and spelling variants; filters by category, price and availability.
- [ ] Holiday mode per store: pause new orders for a date range (for example Eid) with a notice banner, while browsing stays on.
- [ ] Display options per store: Bangla or Western numerals for prices and dates; image alt text taken from product names (editable) for search engines and screen readers.
- [ ] Storefront meets WCAG 2.1 AA for contrast, labels and keyboard use.
- [ ] Business details in the store profile (registered name, address, trade license and DBID numbers), shown in the storefront footer and on invoices; the legal review confirms which fields are required.

Domains

- [ ] Every tenant gets `<slug>.<store domain>` through a wildcard route.
- [ ] Custom domains through Cloudflare for SaaS custom hostnames: the admin shows the CNAME record to add, the platform verifies it and Cloudflare issues the certificate. Recommend `www` plus a redirect from the bare domain.
- [ ] Connect the store domain: wildcard route for store subdomains, the Cloudflare for SaaS fallback origin for client domains, shopper email sender records (SPF, DKIM, DMARC), and Search Console alerts on.
- [ ] Once a custom domain is connected, the store's subdomain permanently (301) redirects to it, and canonical URLs, sitemap and feeds use the custom domain, so search engines never see two copies of the store.

Acceptance criteria

- Two tenants with different themes and languages are served from the same deployment, each on its own domain.
- A cached product page is served without any database query (verified in logs).

Phase 4 — Cart, checkout, orders, promotions and cash on delivery

Goal: the first sellable version: a shopper can order with cash on delivery and the client can process the order to delivery. This is the MVP line. Rough size: 5–6 weeks.

Cart

- [ ] Carts identified by an opaque token (cookie) for guests, linked to the customer after login; expire after inactivity (cron cleanup).
- [ ] Line items reference variants by ID; the server recomputes every total on every change from current prices, promotions and delivery rules. The client never sends a price.
- [ ] Validation on every change: variant active, stock available (unless backorder allowed), quantity limits.

Delivery rates (basic)

- [ ] Delivery zones per tenant built from the global division, district and area reference data (for example inside Dhaka, Dhaka suburbs, outside Dhaka), each with a flat fee and optional weight-based extra.

- [ ] Free delivery is handled by the promotions engine below (thresholds, zones, products, campaigns).

Checkout

- [ ] Phone-first guest checkout: name, phone (validated Bangladeshi format), optional email, address with division, district, area and street.
- [ ] One `placeOrder` command with an idempotency key that, in a single transaction: revalidates the cart, reserves stock with conditional updates, freezes snapshots, creates the order and its lines, creates the payment record, writes `order.placed` to the outbox, and marks the cart converted.
- [ ] Optional fraud basics: order rate limit per phone and IP, a per-tenant blocklist, and an "OTP before COD order, for stores with SMS" toggle.
- [ ] Cloudflare Turnstile (free and usually invisible) on sign-in code requests, checkout, return requests and the contact form, so bots cannot place fake orders or trigger SMS and email codes.

Orders

- [ ] Order status: `pending` → `confirmed` → `processing` → `completed`, plus `cancelled`. Payment, delivery and return progress are separate statuses, each with its own state machine, the way Shopify separates order, payment and fulfillment status: payment (unpaid, partially paid, paid, partially refunded, refunded), delivery (not shipped, partially shipped, shipped, delivered, failed) and returns (none, requested, partially returned, returned). Partial returns and failed deliveries never need extra order states.
- [ ] Status history, internal notes, customer notes, tags.
- [ ] Admin: order list with filters (status, date, payment, delivery zone), order detail, confirm, cancel (releases stock), mark shipped with tracking number, mark delivered, printable invoice and packing slip.
- [ ] Order editing before shipping (change quantity, add or remove items) recalculates totals and adjusts reservations, recorded in history.
- [ ] Customers created or matched by phone within the tenant; customer list and detail with order history; customer import from CSV or Excel (name, phone, email, addresses), matched by phone, for clients moving from spreadsheets or WooCommerce.
- [ ] Manual order entry for Facebook, phone and walk-in orders: search or create the customer by phone, add products with live stock, apply discounts, choose delivery zone and payment method, and save. Uses the same `placeOrder` command as the

storefront, with an order source field (website, Facebook, phone, walk-in, other) for reporting.

- [ ] Admin home dashboard: today's and this month's orders and sales, orders waiting for confirmation, orders to ship, low-stock items, and a 7-day sales trend. Figures come from lightweight summary queries, cached briefly.
- [ ] Shipments: one or more per order, each covering specific lines and quantities, with carrier name, tracking number and its own states (pending, shipped, delivered, failed, returned). Updated by staff in this phase; Phase 7 adds courier booking and automatic status updates on the same records.
- [ ] Shopper accounts (optional for shoppers) on the Better Auth shopper instance, scoped to the store (accounts unique per store, enforced with `tenant_id` and RLS): sign-in by email code, or phone code when the store has SMS configured; login methods switched on per store; session cookie on the store's own domain; rate-limited. Guests track orders with order number and phone.
- [ ] Order export to CSV and Excel, including Steadfast and Pathao bulk-upload formats for booking couriers by hand until Phase 7; bulk printing of invoices and packing slips for selected orders.
- [ ] Shoppers can cancel their own order from the tracking page until it is confirmed; stock is released and staff are notified.

Promotions, discounts and free delivery

- [ ] Sale prices: compare-at price per variant shown as a strikethrough, with optional scheduled start and end dates.
- [ ] One promotion engine for everything: conditions (minimum subtotal, minimum quantity, specific products, categories or collections, delivery zone, first order, customer phone or group, payment method such as extra discount for online payment, date range) and actions (percent off order or items, fixed amount off, buy X get Y, free or discounted delivery).
- [ ] Automatic promotions (no code needed) and coupon codes (one shared code or bulk unique codes), with total and per-customer usage limits, priority, stacking rules (exclusive or combinable) and a maximum discount cap.
- [ ] Free-delivery offers: over a spend threshold, for chosen zones, for chosen products, or for a campaign period.
- [ ] Discounts allocated to each order line and snapshotted on the order, so partial returns refund the correct amount.

- [ ] Usage counted with conditional updates so limits hold when many shoppers check out at once.
- [ ] Storefront shows offers: sale badges, applied discounts in cart, and a progress bar toward free delivery.
- [ ] Admin: promotion builder with a preview of which carts qualify, campaign list with usage and revenue impact.

Cash on delivery

- [ ] COD provider: payment created as `pending` at order placement, `paid` when delivered and collected (manual now, courier-driven in Phase 7).
- [ ] Per-tenant COD settings: enabled, maximum order value, optional advance delivery charge.
- [ ] Orders support several payments, for example an online advance for the delivery charge plus cash for the rest; an order is paid when settled payments cover its total. The advance option goes live once SSLCommerz arrives in Phase 5A.

Acceptance criteria

- Two shoppers ordering the last unit at the same moment: exactly one succeeds (concurrency test).
- Retrying `placeOrder` with the same idempotency key returns the same order and never creates two.
- A full COD order can go from placed to delivered in the admin, with stock and history correct at every step.

Phase 5A — Online payments, notifications and returns

Goal: online payment, shopper and seller notifications, and returns, exchanges and refunds. Rough size: 3–4 weeks.

SSLCommerz provider

- [ ] Each client uses their own SSLCommerz merchant account; credentials stored encrypted per tenant, sandbox and live modes.
- [ ] Flow: create a gateway session server-side, redirect the shopper, receive the IPN webhook, call SSLCommerz's validation API, then verify transaction ID, amount, currency and status before marking the payment `paid`. Redirect pages only display status.

- [ ] Webhook processing is idempotent (unique constraint on the gateway transaction ID) and works even if the shopper closes the browser.
- [ ] A delayed queue message, sent when the payment starts, re-validates it after 30 minutes if it is still pending, and after about 60 minutes cancels the unpaid order and releases its reserved stock (no polling); refunds from the admin through the gateway's refund API.

Notifications

- [ ] Message providers chosen per store. **Email:** the platform's default sender (free tier, store domain, store name as sender, reply-to set to the store's own address), or the store's own email service API key, falling back to the platform sender if it fails. **SMS:** only through the store's own Bangladeshi SMS gateway account in the MVP, with adapters for one or two popular gateways plus a generic HTTP adapter; stores without SMS simply skip SMS features. Platform-provided SMS with prepaid credits comes later.
- [ ] Events: order placed, confirmed, shipped, delivered and cancelled, delivered through the outbox and Queues.
- [ ] Templates per tenant and locale with safe variable substitution; a delivery log with status and retries per message.
- [ ] The free email allowance is shared by all stores: track daily volume against the provider's cap, send transactional messages first, and alert at 70%.
- [ ] New-order alerts for the seller: instant web push to the installed admin app, plus email and optional SMS, chosen per staff member; also alerts for SSLCommerz payments received and return requests.

Returns, exchanges, failed deliveries and refunds

- [ ] Return policy per tenant: return window after delivery, eligible categories, who pays return delivery, accepted refund methods.
- [ ] Return requests created by staff, or by the shopper from the order tracking page: choose lines and quantities, reason (wrong size, damaged, wrong item, not as described, changed mind), optional photos stored in R2.
- [ ] Return state machine: requested, approved or rejected, in transit, received, inspected, resolved (refunded, exchanged, store credit, or rejected after inspection), with full history.
- [ ] Exchanges: a replacement order linked to the return, stock reserved for it, and any price difference charged or refunded.

- [ ] Failed delivery (return to origin): when a COD customer refuses or cannot be reached, mark the fulfillment returned, restock, record the lost delivery cost and flag the phone number. Per-tenant option to require an advance delivery charge from flagged numbers.
- [ ] Restock by inspected condition: sellable items go back to available stock; damaged items are written off with a stock movement reason.
- [ ] Refunds calculated from each line's allocated discount and delivery share. Online payments refund through the SSLCommerz refund API; COD refunds are recorded as manual refunds (bKash, Nagad, bank, cash) with a reference number; store credit is an option.
- [ ] Notifications for return approved, item received, and refund issued.
- [ ] Reports: return rate by product and reason, refund totals, failed-delivery rate and cost by delivery zone.

Acceptance criteria

- SSLCommerz sandbox: payments settle only after validation; forged, replayed and amount-tampered callbacks are rejected by tests.
- A partial return of a discounted order refunds exactly the allocated amount, and stock reflects the inspection result.
- A store without its own SMS gateway runs fully on email and in-app alerts with no errors.

Phase 5B — Launch readiness and pilot

Goal: everything needed to run real clients as a business (onboarding, billing, backups, data export and cleanup), then the pilot. Rough size: 2–3 weeks plus a 4–6 week pilot.

Super-admin onboarding

- [ ] Onboarding screen in the super-admin area: create store, invite owner, choose plan, subdomain and theme preset, with a checklist of missing setup (payments, delivery zones, legal pages, domain, email and SMS). Replaces the CLI for day-to-day onboarding; the CLI stays for automation.
- [ ] Store health view: payment, email, SMS and domain status per store.
- [ ] Platform dashboard for super admins: active stores, orders and sales across stores, resource usage per store (storage, emails, requests), and free-tier usage against each limit with alerts at 70%.

- [ ] Demo store with realistic sample products, orders and a polished theme for sales meetings, reset to clean sample data by the weekly job.
- [ ] Terms acceptance: record which version of the platform terms each client owner accepted and when, and ask again when the terms change.

Platform plans and billing (our charges to clients)

- [ ] Plans stored in the `platform` schema, each with entitlements: limits (products, staff accounts, monthly orders, emails, storage, custom domains) and feature flags (SSLCommerz, promotions types, reports).
- [ ] Entitlements checked in the core services with clear upgrade messages in the admin; nothing breaks for shoppers when a limit is reached.
- [ ] Usage metering per tenant (orders, emails sent, storage, custom domains) rolled up daily by cron.
- [ ] Invoices to clients for setup fees, monthly plans and usage beyond plan limits; payments recorded from bank transfer, bKash, or an SSLCommerz payment link on our own merchant account.
- [ ] Automatic reminders before and after the due date, a grace period, then automatic suspension through the existing tenant lifecycle; reactivation when paid.
- [ ] Super-admin billing screens: plan changes, invoices, payments, overdue list.

Store data export and cleanup

- [ ] Lifecycle: active → suspended (maintenance page, admin read-only, reversible) → closed (storefront and admin off, export generated) → purged after a contractual grace period (default 30 days).
- [ ] Export job: every `store` table as JSON and CSV plus the tenant's R2 files, zipped to R2 with a time-limited download link for the client. The same format is reused later to move a client to a dedicated database.
- [ ] Purge job, super admin only: typed slug confirmation and a dry-run report of row and file counts first; then deletes the tenant's `store` rows in dependency order, in small batches per transaction, resumable after interruption; deletes the R2 prefix, KV keys, custom hostname, stored credentials and pending queue messages; removes store memberships, deleting staff accounts that belonged only to this store.
- [ ] Platform data stays: the tenant registry entry (marked purged), plan and billing history, contract dates, platform audit entries, and a purge record (who approved, when, and row and file counts deleted).

- [ ] Verification after purge: zero rows for the tenant in every `store` table and zero objects under its R2 prefix; the result is stored on the purge record.
- [ ] Shopper deletion requests: anonymize one customer within a store (name, phone, email and addresses replaced with placeholders; order totals kept for the client's sales reports).
- [ ] Store reset: the same purge for demo or test stores, keeping the tenant record and its staff.
- [ ] Terms state the backup retention period (Neon restore history and R2 backups expire on their own schedule and are not edited) and that the export is the client's own record for any legal retention; confirm retention rules with an accountant.

Pilot launch checklist

- [ ] Onboarding playbook and the super-admin onboarding screen tested end to end by a non-developer.
- [ ] Independent backups: a scheduled GitHub Actions workflow runs an encrypted pg_dump of Neon every night and uploads it to R2, keeping 7 daily and 4 weekly copies; a monthly test restores one into a Neon branch. Neon's own restore window on the free plan covers only recent mistakes, and the dumps stay far below R2's free 10 GB. The backup encryption key and the credential encryption key are also kept in a password manager outside Cloudflare, since losing them makes backups and stored client credentials unreadable.
- [ ] Legal pages templates (terms, privacy, returns) per tenant; platform terms and data processing agreement with clients.
- [ ] Security review against the cross-cutting checklist; load test of catalog and checkout against free-tier limits.
- [ ] Runbooks in `docs/runbooks/`: deploy, rollback, restore a backup, rotate a leaked secret, payment or courier outage, a free-tier limit reached, onboarding and offboarding a client.

Acceptance criteria

- A non-developer onboards a new store through the super-admin screen in under an hour, excluding the client's own gateway approvals.
- Pilot clients take real orders for four weeks with no cross-tenant incident and no lost orders, still on $0 infrastructure.
- Purging a seeded test store leaves zero store rows and files for it, leaves the other tenants untouched, and keeps its platform record and billing history.

Phases 6–10 — After the pilot

Each of these starts only when pilot clients need it, in this default order; the agent writes a phase plan for approval before starting each one.

Phase 6 — Sales recovery (about 2–3 weeks, right after the pilot)

- [ ] Incomplete orders: when a shopper enters a phone number at checkout but does not finish, save the cart and contact details (with a short notice at checkout). Staff see an "Incomplete orders" list, can call, and can convert one to an order in a click. Records expire automatically after a set number of days.
- [ ] Fake-order check: a delivery success score per phone number from the store's own history (delivered, failed, returned, cancelled), shown on every order and in manual order entry, with per-store rules such as requiring an advance delivery charge below a score. Sharing scores across stores is off by default and only allowed if platform terms and client consent permit it; courier-provided fraud data can be added in Phase 7.
- [ ] Order confirmation workflow: a confirmation queue for new COD orders, click-to-call, call outcomes (confirmed, no answer, wrong number, cancelled) with retry counts and notes, and an optional SMS link or OTP (stores with SMS) that lets shoppers confirm or cancel themselves. Confirmation rates appear on the dashboard.
- [ ] Duplicate order detection: flag orders from the same phone number with the same items placed within a few minutes, with one-click merge or cancel.

Phase 7 — Couriers and fulfillment (about 3 weeks)

- [ ] Extend the Phase 4 shipment records with courier states (booked, picked up, in transit) driven by courier APIs.
- [ ] `CourierProvider` interface (create parcel, get status, cancel, parse webhook) with adapters for Steadfast, then Pathao, then RedX; credentials per tenant.
- [ ] One-click and bulk booking from the order list; status sync by courier webhooks, with polling only for open shipments a few times a day; delivered status settles COD payments.
- [ ] COD reconciliation report: collected, courier fee, amount paid out, differences.
- [ ] Courier-driven returns: courier return statuses automatically open the failed-delivery and return flows built in Phase 5A.
- [ ] Area mapping: match our division, district and area reference data to each courier's own city, zone and area IDs (pulled from courier APIs and refreshed on a schedule), with a review screen for unmatched areas.

Phase 8 — Back office (about 4–6 weeks)

- [ ] Suppliers, purchase orders, goods receipt (partial and full) updating stock and weighted average cost.
- [ ] Stock adjustments with reasons and stock-count sessions.
- [ ] Expenses with categories and recurring entries.
- [ ] Reports: sales by day and product, gross profit per order and product (cost snapshot on order lines), net profit after expenses, stock valuation, supplier dues, COD outstanding; CSV and Excel export.
- [ ] Reports read from summary tables refreshed by cron to keep queries light.

Phase 9 — Meta and messaging (about 4 weeks)

- [ ] "New order from chat" shortcuts for Messenger and WhatsApp, built on the Phase 4 manual order screen: start an order from a conversation with the customer's details prefilled, and send payment links by chat.
- [ ] Per-tenant product feed for Meta Commerce Manager (catalog for Facebook and Instagram shops and ads), cached in R2.
- [ ] Meta Conversions API (server-side) purchase events, deduplicated against the browser pixel added in Phase 3.
- [ ] WhatsApp Cloud API order notifications through the same notification provider interface, using approved templates.

Phase 10 — Growth (as triggered)

Item	Trigger	Approach
Workers Paid ($5/month)	Any free limit above 70% for a week	Upgrade; no code change
Neon paid plan	Storage near 0.5 GB or compute hours near the free limit	Upgrade; no code change
Price lists, customer-group pricing	Client demand	Already supported by the promotion and pricing models
Better search	Catalogs above roughly 10,000 variants or Bangla search complaints	Dedicated search service behind a `SearchProvider` interface

Item	Trigger	Approach
Dedicated database for a large client	Isolation or volume needs	Same code, separate Neon project; tenant export and import tools
Web POS	Clients with physical shops	Tablet-friendly screen on the same API, sharing inventory
Self-service sign-up and automated billing	Sales model change	Only after onboarding and billing are fully automated

Later features (scheduled by client demand)

- [ ] Product reviews and ratings: verified-purchase reviews, star ratings, photo reviews, moderation in the admin, and review data in product structured data for search results.
- [ ] Back-in-stock alerts: shoppers leave a phone or email on sold-out variants and get an SMS or email when stock returns.
- [ ] Store pickup: pickup locations per store as a delivery option, with a ready-for-pickup status and notification.
- [ ] Blog and content pages: simple posts with categories, SEO fields and product links, to help stores rank in search.
- [ ] Platform SMS: our own gateway account offered to stores without one, paid through prepaid SMS credits per store with low-balance alerts.
- [ ] VAT invoices in the official Mushak 6.3 format for VAT-registered stores.
- [ ] Social login for shoppers (Google, Facebook) through one shared login address, `auth.yourstores.com`: the shopper signs in there and is sent back to the store with a one-time code that the store exchanges for its own session cookie, so one Google app and one Facebook app serve every store. Account linking lets one shopper use several methods. Passkeys for shoppers and staff.
- [ ] Combo packs and bundles: a bundle product made of other variants, priced together, with stock taken from each component.
- [ ] Off-platform backup copy: the weekly encrypted backup also copied to a second free provider (for example Backblaze B2), so backups survive a Cloudflare account problem.

Security, testing and quality (every phase)

Because we own all the commerce logic, correctness of money, stock and tenant isolation is tested on every pull request rather than reviewed once.

Security baseline (OWASP ASVS Level 2 and OWASP Top 10 as the checklist)

- Input validated with zod at every boundary (API, webhooks, imports, queue messages); output encoded; rich text sanitized.
- Authorization checked twice: route permission and service-level tenant and ownership checks.
- Rate limits on auth, OTP, checkout, search and webhooks, plus Cloudflare Turnstile on public forms; CORS limited to known origins; security headers (CSP, HSTS, frame restrictions on admin).
- Webhooks verified by signature or gateway validation API, with replay protection.
- Secrets only in Worker secrets; per-tenant credentials encrypted; no secrets or personal data in logs.
- Dependencies updated by Renovate; `pnpm audit` in CI.
- Card data never touches our systems (SSLCommerz hosted page).

Test strategy

Layer	What it covers	Tool
Unit	Money math, state machines, promotion rules, refund allocation, delivery-fee rules, permission checks	Vitest
Integration	Core services against real Postgres with RLS on: transactions, reservations, idempotency, outbox writes	Vitest + Docker Postgres or Neon branch
Isolation suite	Every tenant-owned table and endpoint, through the API and through raw SQL as the app role	Vitest
Concurrency	Last-unit stock race, coupon usage limits, duplicate webhooks	Vitest with parallel requests
End to end	Browse, cart, checkout (COD and SSLCommerz sandbox), admin order processing, two tenants	Playwright

Layer	What it covers	Tool
Performance	CPU time per request against the 10 ms free limit; page weight budgets	Workers analytics, Lighthouse CI

Observability

- Structured JSON logs with request ID, tenant ID and user ID on every line; Sentry for errors; uptime check on `/health`.
- Alerts: error rate, failed outbox deliveries, payments pending too long, free-tier usage above 70%.

Data protection and compliance

- Collect only data needed to fulfil orders; per-store privacy policy; support deletion requests.
- Follow Bangladesh's Digital Commerce Operation Guidelines for delivery, refund and display rules; local legal review before launch. Clients remain responsible for their own trade license, VAT registration and product legality.

Definition of done for every task

- Typed, Biome-clean, tests added (and isolation tests extended when data access changed).
- Migration included and reviewed if the schema changed, with RLS policy for any new tenant table.
- Bangla and English strings for anything user-facing.
- OpenAPI updated, docs or ADR updated, changelog entry.
- Verified on the preview environment against the phase's acceptance criteria.
- New user-facing screens meet WCAG 2.1 AA for contrast, labels and keyboard use.
- KV only for rarely changing data (domain-to-store map, cache versions): the free plan allows 100,000 reads but only 1,000 writes a day.

Free-tier budget and upgrade triggers

The whole platform runs at $0 until one of these limits is consistently above 70%; the first upgrade (Workers Paid, $5/month) lifts most of them at once.

Service	Free allowance	What it means for us	Upgrade
Cloudflare Workers	100,000 requests/day, 10 ms CPU per request; waiting on database or network does not count as CPU	Roughly a few thousand shopper sessions a day if catalog pages are cached (approximate)	Workers Paid $5/month: 10 million requests/month, CPU up to minutes
Static assets and cache hits	Static asset requests are free and unlimited; cache hits use no CPU	Admin app and cached storefront pages cost nothing	—
Worker size	3 MiB on free, 10 MiB on paid	Keep dependencies lean; check in CI	Workers Paid
Hyperdrive	100,000 database queries/day; cached queries count too	About 15,000 uncached API calls a day at ~6 queries each	Unlimited on Workers Paid
Cloudflare Queues	10,000 operations/day, 24-hour retention	About 3,000 events a day (write, read, delete each count)	1 million operations/month on Workers Paid
Neon Postgres	100 CU-hours and 0.5 GB per project; compute suspends when idle	Serverless API wakes the database only on real traffic	Launch plan, usage-based
Cloudflare for SaaS	100 custom hostnames free, then $0.10 each per month	First 100 client domains free	Pay per hostname
R2	10 GB-month storage, 1 million writes and 10 million reads a month, no egress fees	Product images, exports and nightly database backups	Pay per GB stored
Email, Sentry, uptime monitor, GitHub Actions	Free tiers	Enough for the pilot; the free email cap is shared by all stores	Upgrade per service

Service	Free allowance	What it means for us	Upgrade
SMS, WhatsApp messages	Not provided by us in the MVP	Stores use their own SMS gateway account	Platform SMS with prepaid credits later

Optimization and resource budget

At pilot traffic every free limit has plenty of room except Neon compute time, so the design keeps the database asleep whenever nobody is buying or managing a store, and keeps storage lean without deleting anything the business may need.

Expected pilot usage (assumption: 3 stores, about 300 visitors per store per day, about 20 orders a day in total; measure in Phase 0)

Resource	Free limit	Pilot estimate	Share
Worker requests	100,000/day	~10,000 (most pages served from cache)	~10%
Hyperdrive queries	100,000/day	~10,000–15,000	~10–15%
Queue operations	10,000/day	under 500	~5%
KV writes	1,000/day	a handful	~1%
R2 storage	10 GB	well under 1 GB	~5%
Neon storage	0.5 GB	under 50 MB	~10%
Neon compute	100 CU-hours/month (about 400 awake hours at 0.25 CU)	45–75 CU-hours	45–75%

Steady low traffic is the worst case for Neon: the database sleeps only after 5 minutes without queries, so a few shoppers and staff spread through the day can keep it awake for hours. Neon's paid plan is therefore the first upgrade a busy pilot is likely to need.

Keeping usage low

- Cache all browsing at the edge (catalog pages and public API responses, refreshed in the background), so only cart, checkout and admin actions reach the API Worker and the database. This one rule cuts Worker, Hyperdrive and Neon usage together.
- Nothing on a timer touches the database: outbox events are pushed through Queues as they happen; checks that must happen later (for example re-validating a pending payment after 30 minutes) use delayed queue messages instead of polling; scheduled jobs run once a day in a single batch and exit without a query when there is nothing to do.
- `/health` never touches the database; a deeper check that does runs hourly.
- Neon compute fixed at the smallest size (0.25 CU, autoscaling off), with an alert at 70% of monthly compute hours.
- One queue message per event; its consumer handles every side effect of that event (email, SMS, cache refresh).
- Each request combines its queries into as few round-trips as possible, sets the tenant in the first one, and never runs N+1 queries; every list is paginated.
- Cloudflare's free bot protection on both domains, rate limits on search, and a sensible `robots.txt`, so scrapers do not burn requests.
- Image URLs include a content hash, so they are cached for a year.
- Small dependencies only: built-in `Intl` for dates and numbers, no heavy utility libraries.
- Routine request logs sampled; errors and warnings always kept; Sentry errors sampled and grouped.
- Staff can choose a daily email summary instead of an email per order.
- CI runs tests against a Postgres container inside GitHub Actions; Neon preview branches are created only when a pull request is labeled for it and deleted after merge; dependency caching on, full end-to-end suite only on `main` or labeled pull requests.
- Never put production on a plan whose terms forbid commercial use.

Database storage

- Compact types: `uuid` for IDs, integers for money, small integers or enums for statuses, `timestamptz` for time; order snapshots store only the fields an order needs, not whole product records.
- Lean indexes: tenant ID first in composite indexes, no duplicates; the trigram search index covers names, SKUs and tags only, not descriptions.

- Storage tracked per tenant (row counts and estimated size), for plan limits and to see which client grows fastest.
- Rough capacity: a typical order with lines, payments and history takes a few KB, so 0.5 GB holds on the order of 100,000 orders plus catalogs (estimate; measure in Phase 0).

What is kept and what is cleaned up

Kept for the life of the store: orders, payments, refunds, returns, the stock ledger, the audit log, and payment webhook records (needed for disputes). Cleaned up by the daily job, because it is purely technical:

Data	Deleted after
Expired login sessions and sign-in codes	1 day
Idempotency (retry-protection) keys	2 days
Delivered outbox events	7 days
Abandoned carts	30 days
Incomplete orders (also better for privacy)	30 days
Email and SMS delivery logs	90 days

Archiving is not built now. The audit log and status history are append-only and indexed by time, so they can be moved to compressed files in R2 later; that archiving is built only when database storage passes 60%.

File storage

- The admin app resizes photos in the browser before upload, to WebP at two or three fixed widths (typically 100–200 KB each instead of 3–5 MB); originals are not stored. This costs no server CPU and keeps uploads fast on mobile data.
- Files are named by a hash of their content, so the same image is stored once.
- A weekly job removes images that no product uses any more.
- Product videos are YouTube links, not uploads.
- R2 lifecycle rules, set once in Cloudflare, delete files automatically with no code or jobs:

Folder	Rule
`exports/`	Deleted after 7 days
`tmp-uploads/`	Deleted after 1 day (uploads never attached to a product)
`backups/daily/`	Deleted after 8 days (keeps 7 daily copies)
`backups/weekly/`	Deleted after 35 days (keeps 4 weekly copies)
Product images	No rule; removed by the weekly orphan cleanup when unused

Working instructions for the AI coding agent

This plan is deliberately high level: you own the low-level design, and you are expected to apply established commerce and SaaS practice to every decision it leaves open.

How to work

1. Work phase by phase. At the start of each phase, write `docs/phases/phase-N-plan.md`: the data model (tables, key columns, indexes, RLS policies), API endpoints, state machines, events, screens, tests, and open questions. Wait for approval before coding.
2. Before designing a module, study how Medusa, Vendure, Saleor and Shopify model that domain (their public docs and source), and note in the phase plan what you borrowed and what you simplified, and why.
3. Deliver in pull requests small enough to review (ideally under 400 changed lines), each with tests and a short description.
4. Record any decision about architecture, security, data model, cost or a new dependency as an ADR in `docs/adr/` (context, options, decision, consequences).
5. End each phase with `docs/phases/phase-N-report.md`: what was built, how acceptance criteria were verified, free-tier usage measured, known gaps.

Non-negotiable rules

- Follow the core engineering principles section exactly: tenant isolation in code and RLS, integer money, UTC time, state machines, snapshots, transactions, idempotency, outbox.

- Never trust the client for prices, totals, stock or tenant identity.
- Never settle a payment without server-side gateway validation.
- Never add a paid service, or a dependency that would push a Worker past its size limit, without asking.
- Never commit secrets or real customer data.
- Every user-facing string in Bangla and English.
- Never start a phase until its prerequisites (see Prerequisites by phase) have been provided; list anything missing and ask for it first.
- Never push directly to `main`: open a pull request and merge only after all checks pass.

Conventions

- Modules in `packages/core/<module>` expose commands and queries through a service API; no module imports another module's tables directly.
- REST resources are plural nouns (`/v1/products`, `/v1/orders/{id}/cancel` for commands); cursor pagination, filtering and sorting on every list; errors as RFC 9457 problem details with stable error codes.
- Separate API surfaces: `/v1/store/*` (public storefront, cache-friendly) and `/v1/admin/*` (authenticated staff); platform operations under `/v1/platform/*` for super admins only.
- Database naming: snake_case tables and columns, plural table names, `created_at` and `updated_at` on every table, soft delete only where the domain needs it (products, customers), never on orders or payments.

Questions to ask the product owner before Phase 0

- Platform name, main domain and store domain.
- The first pilot clients, their product types and how they currently take orders.
- Which SMS gateway, email provider and courier the pilots use.
- Who will onboard and support clients, to shape the admin and platform tools.

Sources

- Medusa: Commerce Modules
- Medusa: Commerce features overview
- Vendure: Orders and the order state machine
- Vendure: Fulfillment

- Stripe: Idempotent requests
- Microservices.io: Transactional outbox pattern
- PostgreSQL row-level security for multi-tenant SaaS
- Cloudflare Workers: Limits
- Cloudflare Workers: Pricing
- Cloudflare Hyperdrive: Pricing
- Cloudflare Queues on the Workers Free plan
- Cloudflare for SaaS: Plans
- Cloudflare: Astro on Workers
- Neon: Free plan limits
- OpenNext: Cloudflare (Worker size limits)
- Cloudflare Workers KV: Limits
- Cloudflare R2: Pricing